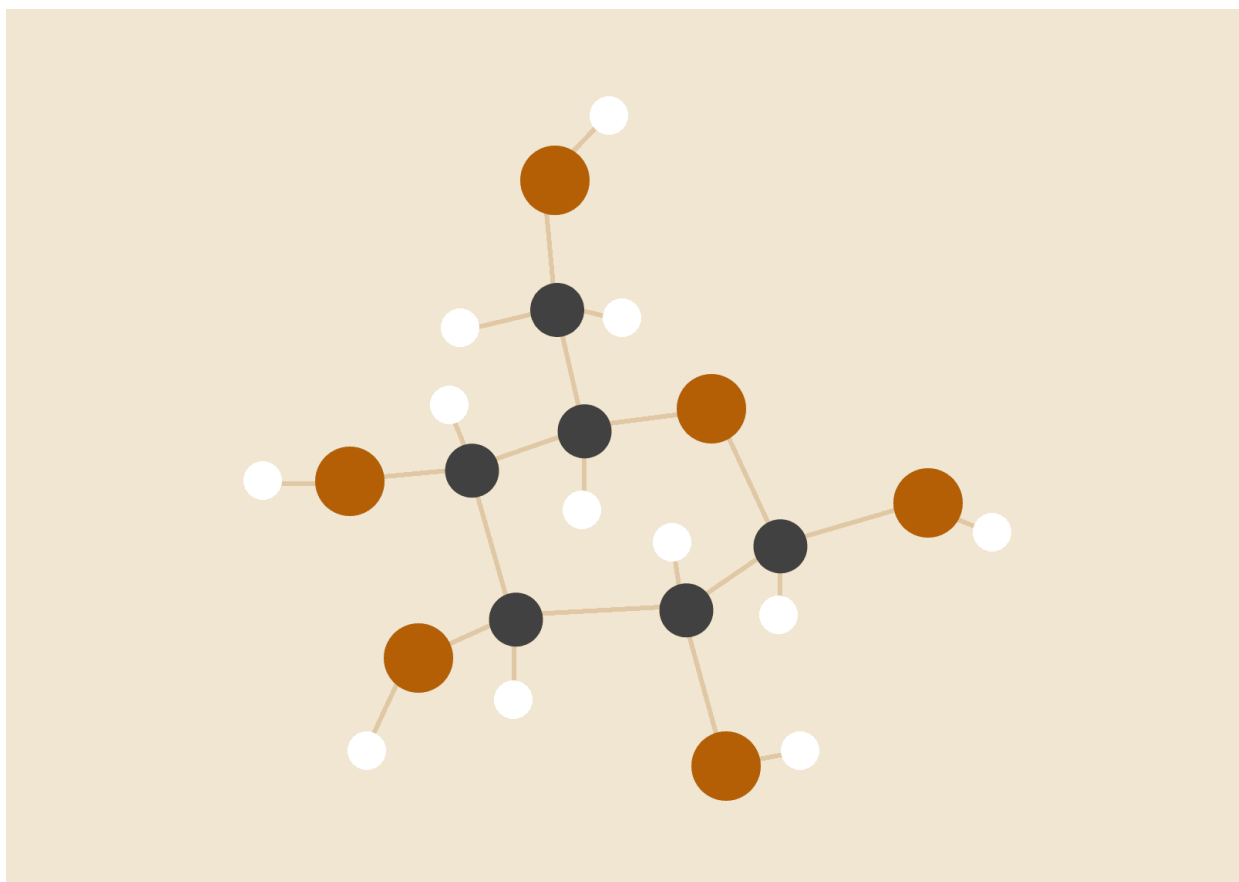


EDD TRACK Manual Tecnico

Fase 2



Fernando Andhre Gonzalez Espinoza

202202055
EDD Sección C

2. Introducción técnica

EDD MedTrack es una aplicación desarrollada en Perl con interfaz gráfica GTK3. Su objetivo es implementar estructuras de datos no lineales y conectarlas a un sistema funcional hospitalario. El sistema administra inventario y personal médico, utilizando JSON para carga masiva y Graphviz para generación de reportes.

3. Objetivo técnico

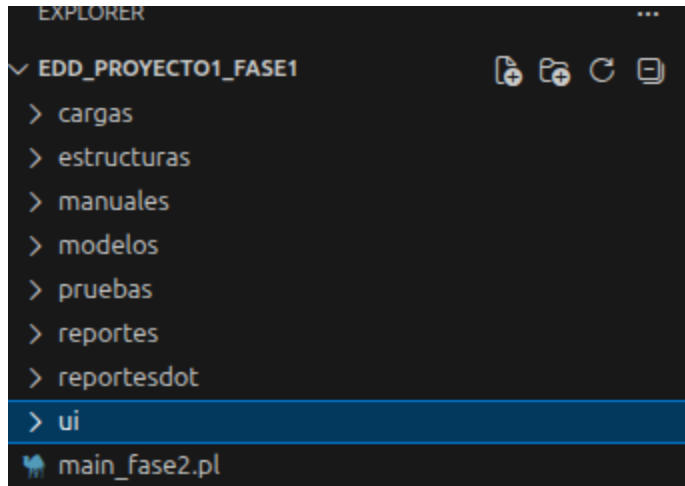
El sistema fue construido para demostrar la integración entre:

- estructuras de datos manuales,
 - interfaz gráfica,
 - procesamiento de archivos JSON,
 - control de acceso,
 - y reportes visuales.
-

4. Arquitectura general

La arquitectura del proyecto se divide en:

- **modelos:** representan entidades del sistema
- **estructuras:** implementan listas, matriz y árboles
- **módulos:** contienen la lógica del negocio
- **ui:** contiene ventanas GTK
- **cargas:** almacena JSON de entrada
- **reportesdot:** almacena archivos `.dot` y `.png`



5. Tecnologías utilizadas

- Perl
 - GTK3
 - Graphviz
 - JSON
 - estructuras de datos implementadas manualmente
-

6. Descripción de modelos

Los modelos representan objetos del dominio hospitalario.

6.1 Medicamento

Contiene atributos como:

- código,
- nombre,
- principio activo,
- laboratorio o fabricante,
- cantidad,
- fecha de vencimiento,
- nivel mínimo.

6.2 Equipo

Contiene:

- código,
- nombre,
- fabricante,
- precio unitario,
- cantidad,
- fecha de ingreso,
- nivel mínimo.

6.3 Suministro

Contiene:

- código,
- nombre,
- fabricante,
- precio unitario,
- cantidad,
- fecha de vencimiento,
- nivel mínimo.

6.4 PersonalMedico

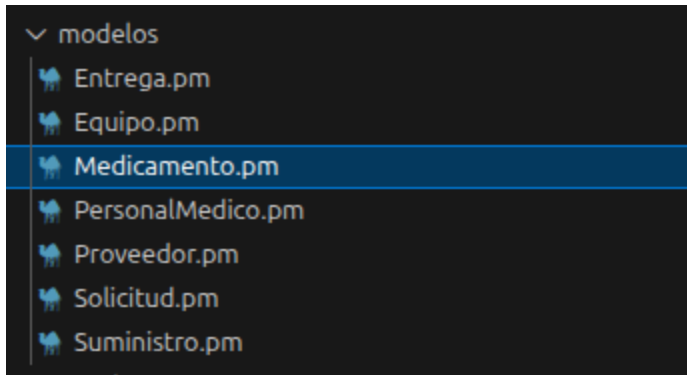
Contiene:

- número de colegio,
- nombre completo,
- tipo de usuario,
- departamento,
- especialidad,
- contraseña.

6.5 Proveedor

Contiene:

- NIT,
- nombre,
- dirección,
- teléfono,
- correo u otros campos definidos.



7. Estructuras de datos implementadas

La hoja de calificación exige explícitamente Matriz Dispersa, BST, AVL y Árbol B.

7.1 Árbol BST

Usado para almacenar el inventario de equipos médicos.

Permite:

- inserción,
- búsqueda,
- eliminación,
- recorridos.

```

estructuras > bst > ♥ NodoBST.pm > {} estructuras::bst::NodoBST
1  package estructuras::bst::NodoBST;
2
3  use strict;
4  use warnings;
5
6  sub new {
7      my ($class, $equipo) = @_;
8
9      my $self = {
10         equipo    => $equipo,
11         izquierdo => undef,
12         derecho   => undef,
13     };
14
15     bless $self, $class;
16     return $self;
17 }
18
19 # =====
20 # Getters
21 # =====
22 sub getEquipo    { return $_[0]->{equipo}; }
23 sub getIzquierdo { return $_[0]->{izquierdo}; }
24 sub getDerecho   { return $_[0]->{derecho}; }
25
26 sub getClave {
27     my ($self) = @_;
28     return $self->{equipo}->getClave();
29 }
30
31 # =====
32 # Setters
33 # =====
34 sub setEquipo {
35     my ($self, $equipo) = @_;
36     $self->{equipo} = $equipo;
37 }
38
39 sub setIzquierdo {
40     my ($self, $nodo) = @_;
41     $self->{izquierdo} = $nodo;
42 }
43
44 sub setDerecho {
45     my ($self, $nodo) = @_;
46     $self->{derecho} = $nodo;
47 }
48
49 1;

```

```

estructuras > bst > ♥ ArbolBST.pm > {} estructuras::bst::ArbolBST > 📄 new
1  package estructuras::bst::ArbolBST;
2
3  use strict;
4  use warnings;
5  use estructuras::bst::NodoBST;
6
7  sub new {
8      my ($class) = @_ ;
9
10     my $self = {
11         raiz => undef,
12         size => 0,
13     };
14
15     bless $self, $class;
16     return $self;
17 }
18
19 # =====
20 # Getters básicos
21 # =====
22 sub getRaiz {
23     my ($self) = @_ ;
24     return $self->{raiz};
25 }
26
27 sub estaVacio {
28     my ($self) = @_ ;
29     return !defined $self->{raiz};
30 }
31
32 sub getSize {
33     my ($self) = @_ ;
34     return $self->{size};
35 }
36
37 # =====
38 # Inserción
39 # =====
40 sub insertar {
41     my ($self, $equipo) = @_ ;
42
43     if (!defined $equipo) {
44         return (0, "No se puede insertar un equipo indefinido");
45     }
46
47     if (!$equipo->can('getClave')) {
48         return (0, "El objeto no tiene metodo getClave");
49     }
50

```

```

estructuras > bst > ArbolBST.pm > {} estructuras::bst::ArbolBST > new
1  package estructuras::bst::ArbolBST;
40 sub insertar {
41     if ($equipo->can( 'evaluar' ) || $equipo->evaluar()) {
42         my @errores = $equipo->validar();
43         return (0, "Equipo invalido: " . join(" ", @errores));
44     }
45
46     my ($nueva_raiz, $insertado, $mensaje) = $self->_insertar_recursivo($self->(raiz), $equipo);
47     $self->(raiz) = $nueva_raiz;
48
49     if ($insertado) {
50         $self->(size)++;
51     }
52
53     return ($insertado, $mensaje);
54 }
55
56 sub _insertar_recursivo {
57     my ($self, $nodo_actual, $equipo) = @_;
58
59     if (!defined $nodo_actual) {
60         my $nuevo_nodo = estructuras::bst::NodoBST->new($equipo);
61         return ($nuevo_nodo, 1, "Equipo insertado correctamente");
62     }
63
64     my $clave_nueva = $equipo->getClave();
65     my $clave_actual = $nodo_actual->getClave();
66
67     if ($clave_nueva lt $clave_actual) {
68         my ($nuevo_izq, $insertado, $mensaje) =
69             $self->_insertar_recursivo($nodo_actual->getIzquierdo(), $equipo);
70         $nodo_actual->setIzquierdo($nuevo_izq);
71         return ($nodo_actual, $insertado, $mensaje);
72     }
73     elsif ($clave_nueva gt $clave_actual) {
74         my ($nuevo_der, $insertado, $mensaje) =
75             $self->_insertar_recursivo($nodo_actual->getDerecho(), $equipo);
76         $nodo_actual->setDerecho($nuevo_der);
77         return ($nodo_actual, $insertado, $mensaje);
78     }
79     else {
80         return ($nodo_actual, 0, "Ya existe un equipo con el codigo $clave_nueva");
81     }
82 }
83
84 # =====
85 # Búsqueda
86 # =====
87 sub buscar {
88     my ($self, $codigo) = @_;
89     my $nodo = $self->buscar_nodo($self->(raiz), $codigo);
90 }

```



```

estructuras > bst > ArbolBST.pm > {} estructuras::bst::ArbolBST > new
1  package estructuras::bst::ArbolBST;
99  sub buscar {
100  |   my ($self, $codigo) = @_;
101  |   my $nodo = $self->_buscar_nodo($self->{raiz}, $codigo);
102  |   return defined $nodo ? $nodo->getEquipo() : undef;
103  | }
104
105  sub _buscar_nodo {
106  |   my ($self, $nodo_actual, $codigo) = @_;
107  |
108  |   return undef if !defined $nodo_actual;
109  |
110  |   my $clave_actual = $nodo_actual->getClave();
111  |
112  |   if ($codigo eq $clave_actual) {
113  |       | return $nodo_actual;
114  |   }
115  |   elsif ($codigo lt $clave_actual) {
116  |       | return $self->_buscar_nodo($nodo_actual->getIzquierdo(), $codigo);
117  |   }
118  |   else {
119  |       | return $self->_buscar_nodo($nodo_actual->getDerecho(), $codigo);
120  |   }
121  | }
122
123  sub existe {
124  |   my ($self, $codigo) = @_;
125  |   return defined $self->buscar($codigo);
126  | }
127
128  # =====
129  # Recorridos
130  # =====
131  sub inOrden {
132  |   my ($self) = @_;
133  |   my @resultado;
134  |   $self->_in_orden_recursivo($self->{raiz}, \@resultado);
135  |   return \@resultado;
136  | }
137
138  sub _in_orden_recursivo {
139  |   my ($self, $nodo, $resultado) = @_;
140  |   return if !defined $nodo;
141  |
142  |   $self->_in_orden_recursivo($nodo->getIzquierdo(), $resultado);
143  |   push @$resultado, $nodo->getEquipo();
144  |   $self->_in_orden_recursivo($nodo->getDerecho(), $resultado);
145  | }
146
147  sub preOrden {
148  |   my ($self) = @_;
149  |   my @resultado;

```

7.2 Árbol AVL

Usado para almacenar el personal médico autorizado.

Permite:

- balanceo automático,

- inserción,
- búsqueda,
- eliminación,
- recorridos.

```

estructuras > avl > ArbolAVL.pm > {} estructuras::avl::ArbolAVL
1  package estructuras::avl::ArbolAVL;
2
3  use strict;
4  use warnings;
5  use estructuras::avl::NodoAVL;
6
7  sub new {
8      my ($class) = @_;
9
10     my $self = {
11         raiz => undef,
12         size => 0,
13     };
14
15     bless $self, $class;
16     return $self;
17 }
18
19 # =====
20 # Getters básicos
21 # =====
22 sub getRaiz {
23     my ($self) = @_;
24     return $self->{raiz};
25 }
26
27 sub estaVacio {
28     my ($self) = @_;
29     return !defined $self->{raiz};
30 }
31
32 sub getSize {
33     my ($self) = @_;
34     return $self->{size};
35 }
36
37 # =====
38 # Utilidades AVL
39 # =====
40 sub _altura {
41     my ($self, $nodo) = @_;
42     return defined $nodo ? $nodo->getAltura() : 0;
43 }
44
45 sub _max {
46     my ($self, $a, $b) = @_;
47     return ($a > $b) ? $a : $b;
48 }
49
50 sub _actualizar_altura {
51     my ($self, $nodo) = @_;
52     return if !defined $nodo;

```

estructuras > avl > ArbolAVL.pm > {} estructuras:avl:ArbolAVL

```
1  package estructuras::avl:ArbolAVL;
50 sub _actualizar_altura {
53
54     my $altura_izq = $self->_altura($nodo->getIzquierdo());
55     my $altura_der = $self->_altura($nodo->getDerecho());
56
57     $nodo->setAltura(1 + $self->_max($altura_izq, $altura_der));
58 }
59
60 sub _obtener_balance {
61     my ($self, $nodo) = @_;
62     return 0 if !defined $nodo;
63
64     return $self->_altura($nodo->getIzquierdo()) - $self->_altura($nodo->getDerecho());
65 }
66
67 # =====
68 # Rotaciones
69 # =====
70 sub _rotacion_derecha {
71     my ($self, $y) = @_;
72
73     my $x = $y->getIzquierdo();
74     my $t2 = $x->getDerecho();
75
76     $x->setDerecho($y);
77     $y->setIzquierdo($t2);
78
79     $self->_actualizar_altura($y);
80     $self->_actualizar_altura($x);
81
82     return $x;
83 }
84
85 sub _rotacion_izquierda {
86     my ($self, $x) = @_;
87
88     my $y = $x->getDerecho();
89     my $t2 = $y->getIzquierdo();
90
91     $y->setIzquierdo($x);
92     $x->setDerecho($t2);
93
94     $self->_actualizar_altura($x);
95     $self->_actualizar_altura($y);
96
97     return $y;
98 }
99
100 # =====
101 # Inserción
102 # =====
```

```

estructuras > avl > ArbolAVL.pm > {} estructuras:avl::ArbolAVL
1  package estructuras::avl::ArbolAVL;
102 # =====
103 sub insertar {
104     my ($self, $personal) = @_;
105
106     if (!defined $personal) {
107         return (0, "No se puede insertar un usuario indefinido");
108     }
109
110     if (!$personal->can('getClave')) {
111         return (0, "El objeto no tiene metodo getClave");
112     }
113
114     if ($personal->can('esValido') && !$personal->esValido()) {
115         my @errores = $personal->validar();
116         return (0, "Usuario invalido: " . join(", ", @errores));
117     }
118
119     my ($nueva_raiz, $insertado, $mensaje) = $self->_insertar_recursivo($self->{raiz}, $personal);
120     $self->{raiz} = $nueva_raiz;
121
122     if ($insertado) {
123         $self->{size}++;
124     }
125
126     return ($insertado, $mensaje);
127 }
128
129 sub _insertar_recursivo {
130     my ($self, $nodo, $personal) = @_;
131
132     if (!defined $nodo) {
133         my $nuevo = estructuras::avl::NodoAVL->new($personal);
134         return ($nuevo, 1, "Usuario insertado correctamente");
135     }
136
137     my $clave_nueva = $personal->getClave();
138     my $clave_actual = $nodo->getClave();
139
140     if ($clave_nueva lt $clave_actual) {
141         my ($nuevo_izq, $insertado, $mensaje) =
142             $self->_insertar_recursivo($nodo->getIzquierdo(), $personal);
143
144         $nodo->setIzquierdo($nuevo_izq);
145
146         return ($nodo, $insertado, $mensaje) if !$insertado;
147     } elsif ($clave_nueva gt $clave_actual) {
148         my ($nuevo_der, $insertado, $mensaje) =
149             $self->_insertar_recursivo($nodo->getDerecho(), $personal);
150
151         $nodo->setDerecho($nuevo_der);
152     }

```

7.3 Árbol B de orden 4

Usado para almacenar el inventario de suministros.

Permite:

- inserción,

- búsqueda,
- eliminación,
- múltiples claves por nodo.

```

estructuras > arbolB > ArbolB.pm > {} estructuras::arbolB::ArbolB
1  package estructuras::arbolB::ArbolB;
2
3  use strict;
4  use warnings;
5  use estructuras::arbolB::NodoB;
6
7  sub new {
8      my ($class) = @_;
9
10     my $self = {
11         raiz      => estructuras::arbolB::NodoB->new(hoja => 1),
12         orden     => 4,
13         grado_minimo => 2,
14         max_claves => 3,
15         size      => 0,
16     };
17
18     bless $self, $class;
19     return $self;
20 }
21
22 # =====
23 # Getters básicos
24 # =====
25 sub getRaiz {
26     return $_[0]->{raiz};
27 }
28
29 sub getSize {
30     return $_[0]->{size};
31 }
32
33 sub estaVacio {
34     return $_[0]->{size} == 0;
35 }
36
37 sub getOrden {
38     return $_[0]->{orden};
39 }
40
41 # =====
42 # Búsqueda
43 # =====
44 sub buscar {
45     my ($self, $clave) = @_;
46
47     return undef if !defined $clave || $clave eq '';
48     return $self->_buscar_en_nodo($self->{raiz}, $clave);
49 }
50
51 sub _buscar_en_nodo {
52     my ($self, $nodo, $clave) = @_;

```

```

estructuras > arbolB > ♥ ArbolB.pm > {} estructuras::arbolB::ArbolB
1  package estructuras::arbolB::ArbolB;
51 sub _buscar_en_nodo {
53
54     my $i = 0;
55
56     while ($i < $nodo->cantidadClaves() && $clave gt $nodo->getClaveEn($i)) {
57         $i++;
58     }
59
60     if ($i < $nodo->cantidadClaves() && $clave eq $nodo->getClaveEn($i)) {
61         return $nodo->getValorEn($i);
62     }
63
64     return undef if $nodo->esHoja();
65
66     return $self->_buscar_en_nodo($nodo->getHijoEn($i), $clave);
67 }
68
69 sub existe {
70     my ($self, $clave) = @_;
71     return defined $self->buscar($clave);
72 }
73
74 # =====
75 # Inserción
76 # =====
77 sub insertar {
78     my ($self, $obj) = @_;
79
80     if (!defined $obj) {
81         return (0, 'No se puede insertar un suministro indefinido');
82     }
83
84     if (!$obj->can('getClave')) {
85         return (0, 'El objeto no tiene metodo getClave');
86     }
87
88     if ($obj->can('esValido') && !$obj->esValido()) {
89         my @errores = $obj->validar();
90         return (0, 'Suministro invalido: ' . join(', ', @errores));
91     }
92
93     my $clave = $obj->getClave();
94
95     if ($self->existe($clave)) {
96         return (0, "Ya existe un suministro con codigo $clave");
97     }
98
99     my $raiz = $self->{raiz};
100
101     if ($raiz->cantidadClaves() == $self->{max_claves}) {
102         my $nueva_raiz = estructuras::arbolB::NodoB->new(hoja => 0);

```

```

estructuras > arbolB > ♥ ArbolB.pm > {} estructuras::arbolB::ArbolB
1  package estructuras::arbolB::ArbolB;
77  sub insertar {
103      $nueva_raiz->insertarHijoEn(0, $raiz);
104
105      $self->_dividir_hijo($nueva_raiz, 0);
106      $self->{raiz} = $nueva_raiz;
107  }
108
109      $self->_insertar_no_lleno($self->{raiz}, $obj);
110      $self->{size}++;
111
112      return (1, 'Suministro insertado correctamente');
113  }
114
115  sub _insertar_no_lleno {
116      my ($self, $nodo, $obj) = @_;
117
118      my $clave = $obj->getClave();
119      my $i = $nodo->cantidadClaves() - 1;
120
121      if ($nodo->esHoja()) {
122          while ($i >= 0 && $clave lt $nodo->getClaveEn($i)) {
123              $i--;
124          }
125
126          $nodo->insertarClaveValorEn($i + 1, $clave, $obj);
127          return;
128      }
129
130      while ($i >= 0 && $clave lt $nodo->getClaveEn($i)) {
131          $i--;
132      }
133
134      $i++;
135
136      my $hijo = $nodo->getHijoEn($i);
137
138      if ($hijo->cantidadClaves() == $self->{max_claves}) {
139          $self->_dividir_hijo($nodo, $i);
140
141          if ($clave gt $nodo->getClaveEn($i)) {
142              $i++;
143          }
144      }
145
146      $self->_insertar_no_lleno($nodo->getHijoEn($i), $obj);
147  }
148
149  sub _dividir_hijo {
150      my ($self, $padre, $idx_hijo) = @_;
151
152      my $t = $self->{grado_minimo};

```

7.4 Matriz Dispersa

Usada para representar la relación entre proveedores y fabricantes.

estructuras > matrizDispersa >  MatrizDispersa.pm > {} estructuras::matrizDispersa::MatrizDispersa

```
1 package estructuras::matrizDispersa::MatrizDispersa;
2
3 use estructuras::matrizDispersa::ListaCabecera;
4 use estructuras::matrizDispersa::NodoCelda;
5
6 sub new {
7     my ($class) = @_ ;
8
9     my $self = {
10         filas => estructuras::matrizDispersa::ListaCabecera->new(),
11         columnas => estructuras::matrizDispersa::ListaCabecera->new()
12     };
13
14     bless $self, $class;
15     return $self;
16 }
17
18 sub insertar {
19     my ($self, $medicamento, $laboratorio, $precio, $cantidad) = @_;
20
21     # Buscar cabeceras
22     my $cabFila = $self->{filas}->insertarCabecera($medicamento);
23     my $cabCol = $self->{columnas}->insertarCabecera($laboratorio);
24
25     # Verificar si la celda ya existe
26     my $actual = $cabFila->{acceso};
27     while ($actual) {
28         if ($actual->{laboratorio} eq $laboratorio) {
29             $actual->{precio} = $precio;
30             $actual->{cantidad} = $cantidad;
31             return;
32         }
33         $actual = $actual->{derecha};
34     }
35
36     # Crear nueva celda
37     my $nuevo = estructuras::matrizDispersa::NodoCelda
38         ->new($medicamento, $laboratorio, $precio, $cantidad);
39
40     # ---- INSERTAR EN FILA ----
41     if (!$cabFila->{acceso}) {
42         $cabFila->{acceso} = $nuevo;
43     } else {
44         my $actual = $cabFila->{acceso};
45         if ($laboratorio lt $actual->{laboratorio}) {
46             $nuevo->{derecha} = $actual;
47             $actual->{izquierda} = $nuevo;
48             $cabFila->{acceso} = $nuevo;
49         } else {
50             while ($actual->{derecha}
51                 && $laboratorio gt $actual->{derecha}->{laboratorio}) {
52                 $actual = $actual->{derecha};
53             }
54         }
55     }
56 }
```

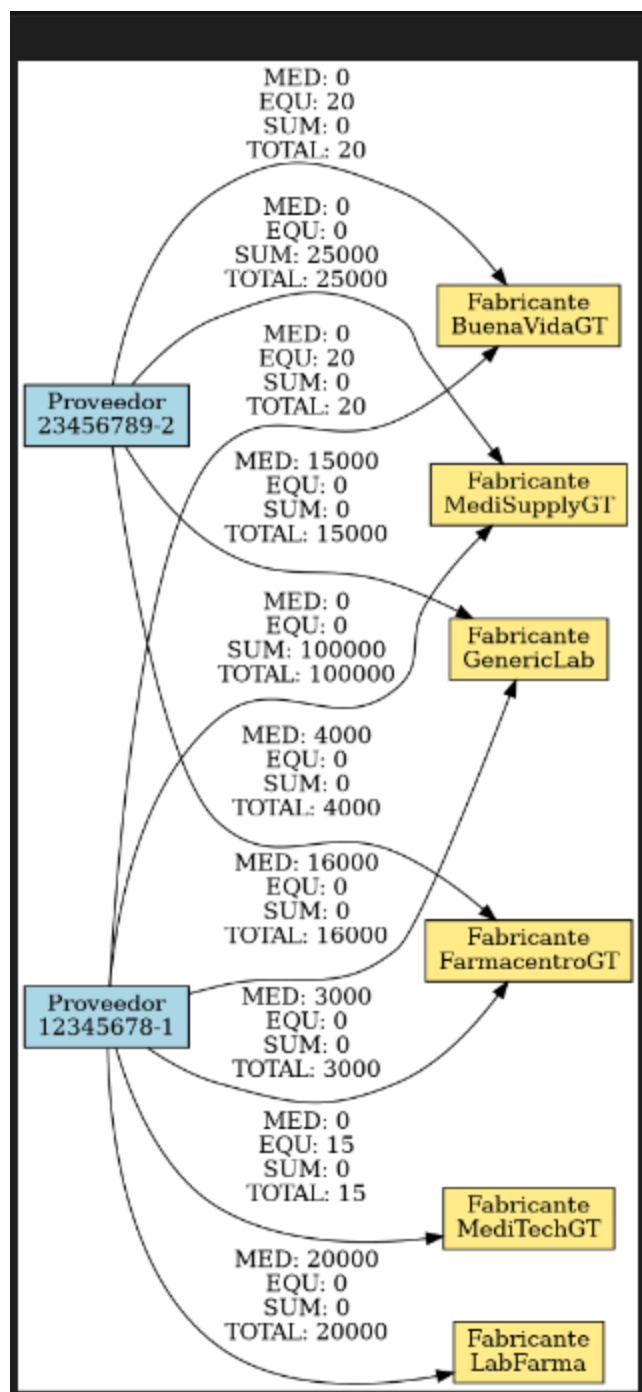

estructuras > matrizDispersa >  MatrizDispersa.pm > {} estructuras::matrizDispersa::MatrizDispersa

```
1  package estructuras::matrizDispersa::MatrizDispersa;
17 sub insertar {
53
54     $nuevo->{derecha} = $actual->{derecha};
55     $nuevo->{izquierda} = $actual;
56
57     $actual->{derecha}->{izquierda} = $nuevo if $actual->{derecha};
58     $actual->{derecha} = $nuevo;
59 }
60 }
61
62 # ---- INSERTAR EN COLUMNA ----
63 if (!$cabCol->{acceso}) {
64     $cabCol->{acceso} = $nuevo;
65 } else {
66     my $actual = $cabCol->{acceso};
67     if ($medicamento lt $actual->{medicamento}) {
68         $nuevo->{abajo} = $actual;
69         $actual->{arriba} = $nuevo;
70         $cabCol->{acceso} = $nuevo;
71     } else {
72         while ($actual->{abajo}
73             && $medicamento gt $actual->{abajo}->{medicamento}) {
74             $actual = $actual->{abajo};
75         }
76
77         $nuevo->{abajo} = $actual->{abajo};
78         $nuevo->{arriba} = $actual;
79
80         $actual->{abajo}->{arriba} = $nuevo if $actual->{abajo};
81         $actual->{abajo} = $nuevo;
82     }
83 }
84 }
85 sub consultarMedicamento {
86     my ($self, $medicamento) = @_;
87
88     my $cabFila = $self->{filas}->obtenerCabecera($medicamento);
89     return unless $cabFila;
90
91     my $actual = $cabFila->{acceso};
92
93     while ($actual) {
94         print "Laboratorio: ", $actual->{laboratorio}, "\n";
95         print "Precio: Q", $actual->{precio}, "\n";
96         print "Cantidad: ", $actual->{cantidad}, "\n";
97         print "-----\n";
98
99         $actual = $actual->{derecha};
100     }
101 }
102 }
```

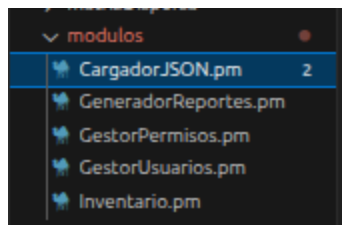
```

estructuras > matrizDispersa > ♥ MatrizDispersa.pm > {} estructuras::matrizDispersa::MatrizDispersa
1  package estructuras::matrizDispersa::MatrizDispersa;
102
103 sub generarDot {
104     my ($self, $ruta_dot) = @_;
105
106     open(my $fh, ">", $ruta_dot) or die "No se pudo crear el archivo DOT";
107
108     print $fh "digraph G {\n";
109     print $fh "rankdir=TB;\n";
110     print $fh "graph [pad=\"0.6\", nodesep=\"0.8\", ranksep=\"1\"];\n";
111     print $fh "node [shape=box, height=0.8, fontname=\"Arial\"];\n";
112     print $fh "edge [fontname=\"Arial\"];\n\n";
113
114     my $clean = sub {
115         my $id = shift;
116         $id =~ s/\s+/_/g;
117         $id =~ s/[^A-Za-z0-9_]/_/g;
118         return $id;
119     };
120
121
122     # 1. Raiz de la matriz
123
124     print $fh "\tRaiz [label=\"\" width=0.3 shape=point];\n";
125
126     # 2. COLUMNAS ARRIBA
127     my $colActual = $self->{columnas}->{primero};
128     my @columnasIDs;
129     my %grupoColumna;
130     my $grupo = 1;
131
132     while ($colActual) {
133
134         my $colID = "Col_" . $clean->{$colActual->{id}};
135         push @columnasIDs, $colID;
136         $grupoColumna{$colActual->{id}} = $grupo;
137
138         print $fh "\t$colID [label=\"$colActual->{id}\" group=$grupo];\n";
139
140         $grupo++;
141         $colActual = $colActual->{siguiente};
142     }
143
144     # Alinear columnas arriba
145     print $fh "\t{ rank=same; Raiz; " . join(" ", @columnasIDs) . "; }\n";
146
147     # Conexión horizontal columnas
148     for (my $i = 0; $i < @columnasIDs - 1; $i++) {
149         print $fh "\t$columnasIDs[$i] -> $columnasIDs[$i+1] [dir=both];\n";
150     }
151
152     print $fh "\tRaiz -> $columnasIDs[0] [dir=both];\n\n"

```



8. Lógica del sistema



8.1 Inventario.pm

Módulo central que coordina:

- medicamentos,
- equipos,
- suministros,
- proveedores,
- matriz resumen,
- generación de reportes,
- comparación proveedor/fabricante.

```

estructuras > modulos >  Inventario.pm > {} estructuras::modulos::Inventario
1  package estructuras::modulos::Inventario;
2
3  use strict;
4  use warnings;
5
6  use File::Path qw(make_path);
7
8  use estructuras::listaDoble::ListaDoble;
9  use estructuras::listaDobleCircular::ListaDobleCircular;
10 use estructuras::matrizDispersa::MatrizDispersa;
11 use estructuras::bst::ArbolBST;
12 use estructuras::arbolB::ArbolB;
13
14 sub new {
15     my ($class, @args) = @_ ;
16
17     my %args;
18
19     if (@args == 1 && ref($args[0]) eq 'HASH') {
20         %args = %{ $args[0] };
21     }
22     elsif (@args >= 2 && ref($args[0])) {
23         $args{medicamentos} = $args[0];
24         $args{proveedores} = $args[1];
25     }
26     else {
27         %args = @args;
28     }
29
30     my $self = {
31         medicamentos => $args{medicamentos} // estructuras::listaDoble::ListaDoble->new(),
32         equipos      => $args{equipos}      // estructuras::bst::ArbolBST->new(),
33         suministros  => $args{suministros}   // estructuras::arbolB::ArbolB->new(),
34         proveedores  => $args{proveedores}   // estructuras::listaDobleCircular::ListaDobleCircular->new(),
35         matriz       => $args{matriz}        // estructuras::matrizDispersa::MatrizDispersa->new(),
36
37         indice_medicamentos => {},
38         indice_proveedores  => {},
39
40         orden_medicamentos => [],
41         orden_proveedores  => [],
42
43         resumen_matriz => {},
44     };
45
46     bless $self, $class;
47     return $self;

```

```

estructuras > modulos > 📁 Inventario.pm > {} estructuras::modulos::Inventario
1  package estructuras::modulos::Inventario;
48 }
49
50 # =====
51 # Getters de estructuras
52 # =====
53 sub getMedicamentos { return $_[0]->{medicamentos}; }
54 sub getEquipos      { return $_[0]->{equipos}; }
55 sub getSuministros  { return $_[0]->{suministros}; }
56 sub getProveedores  { return $_[0]->{proveedores}; }
57 sub getMatriz       { return $_[0]->{matriz}; }
58
59 # =====
60 # Registro de proveedores
61 # =====
62 sub registrarProveedor {
63     my ($self, $proveedor) = @_;
64
65     if (!defined $proveedor) {
66         return (0, 'No se puede registrar un proveedor indefinido');
67     }
68
69     my $nit = $self->_obtener_campo($proveedor, 'nit');
70     return (0, 'El proveedor no tiene NIT') if !defined $nit || $nit eq '';
71
72     if (exists $self->{indice_proveedores}{$nit}) {
73         return (0, "Ya existe un proveedor con NIT $nit");
74     }
75
76     my ($ok, $msg) = $self->_insertar_en_lista($self->{proveedores}, $proveedor);
77     return ($ok, $msg) if !$ok;
78
79     $self->{indice_proveedores}{$nit} = $proveedor;
80     push @{$self->{orden_proveedores}}, $nit;
81
82     return (1, 'Proveedor registrado correctamente');
83 }
84
85 sub buscarProveedor {
86     my ($self, $nit) = @_;
87     return $self->{indice_proveedores}{$nit};
88 }
89
90 sub listarProveedores {
91     my ($self) = @_;
92
93     my @lista;

```

```

estructuras > modulos > Inventario.pm > {} estructuras::modulos::Inventario
1  package estructuras::modulos::Inventario;
90 sub listarProveedores {
91     my ($self) = @_;
92
93     my @lista;
94     foreach my $nit (@{ $self->{orden_proveedores} }) {
95         next if !exists $self->{indice_proveedores}{$nit};
96         push @lista, $self->{indice_proveedores}{$nit};
97     }
98
99     return \@lista;
100 }
101
102 sub eliminarProveedor {
103     my ($self, $nit) = @_;
104
105     return (0, 'Debe indicar el NIT del proveedor') if !defined $nit || $nit eq '';
106     return (0, "No existe un proveedor con NIT $nit") if !exists $self->{indice_proveedores}{$nit};
107
108     my $proveedor = delete $self->{indice_proveedores}{$nit};
109     $self->eliminar_de_lista($self->{proveedores}, $nit, $proveedor);
110
111     @{ $self->{orden_proveedores} } = grep { $_ ne $nit } @{ $self->{orden_proveedores} };
112
113     delete $self->{resumen_matriz}{$nit};
114
115     return (1, 'Proveedor eliminado correctamente');
116 }
117
118 # =====
119 # Registro de medicamentos
120 # =====
121 sub registrarMedicamento {
122     my ($self, $medicamento, $nit_proveedor) = @_;
123
124     if (!defined $medicamento) {
125         return (0, 'No se puede registrar un medicamento indefinido');
126     }
127
128     my $codigo = $self->obtener_campo($medicamento, 'codigo');
129     return (0, 'El medicamento no tiene codigo') if !defined $codigo || $codigo eq '';
130
131     if (exists $self->{indice_medamentos}{$codigo}) {
132         return (0, "Ya existe un medicamento con codigo $codigo");
133     }
134
135     my ($ok, $msg) = $self->insertar_en_lista($self->{medicamentos}, $medicamento);
136 }

```

8.2 GestorUsuarios.pm

Administra:

- registro de usuarios,
- autenticación,
- búsqueda,
- eliminación,
- recorridos,

- Resumen de usuarios.

```
estructuras > modulos >  GestorUsuarios.pm > {} estructuras::modulos::GestorUsuarios
1  package estructuras::modulos::GestorUsuarios;
2
3  use strict;
4  use warnings;
5
6  use File::Path qw(make_path);
7
8  use estructuras::avl::ArbolAVL;
9  use modelos::PersonalMedico;
10
11 sub new {
12     my ($class, @args) = @_;
13
14     my %args;
15
16     # Compatibilidad:
17     # 1) new({ ... })
18     # 2) new(clave => valor, ...)
19     # 3) new($arbol avl)
20     if (@args == 1 && ref($args[0]) eq 'HASH') {
21         %args = %{ $args[0] };
22     }
23     elsif (@args == 1 && ref($args[0])) {
24         $args{arbol} = $args[0];
25     }
26     else {
27         %args = @args;
28     }
29
30     my $self = {
31         arbol => $args{arbol} // estructuras::avl::ArbolAVL->new(),
32     };
33
34     bless $self, $class;
35     return $self;
36 }
37
38 # =====
39 # Getters básicos
40 # =====
41 sub getArbol {
42     my ($self) = @_;
43     return $self->{arbol};
44 }
45
46 sub getCantidadUsuarios {
47     my ($self) = @_;
```



```

estructuras > modulos > ✎ GestorUsuarios.pm > {} estructuras::modulos::GestorUsuarios
1  package estructuras::modulos::GestorUsuarios;
46 sub getCantidadUsuarios {
49 }
50
51 sub estaVacio {
52     my ($self) = @_;
53     return $self->{arbol}->estaVacio();
54 }
55
56 # =====
57 # Registro de usuarios
58 # =====
59 sub registrarUsuario {
60     my ($self, $usuario) = @_;
61
62     if (!defined $usuario) {
63         return (0, 'No se puede registrar un usuario indefinido');
64     }
65
66     if (!$usuario->can('getClave')) {
67         return (0, 'El objeto no tiene metodo getClave');
68     }
69
70     my ($ok, $msg) = $self->{arbol}->insertar($usuario);
71     return ($ok, $msg);
72 }
73
74 sub registrarUsuarioDesdeHash {
75     my ($self, $data) = @_;
76
77     if (!defined $data || ref($data) ne 'HASH') {
78         return (0, 'Debe proporcionar un HASH con los datos del usuario');
79     }
80
81     my $usuario = modelos::PersonalMedico->new(%$data);
82     return $self->registrarUsuario($usuario);
83 }
84
85 # =====
86 # Búsqueda
87 # =====
88 sub buscarUsuario {
89     my ($self, $numero_colegio) = @_;
90
91     return undef if !defined $numero_colegio || $numero_colegio eq '';
92     return $self->{arbol}->buscar($numero_colegio);
93 }

```

```

estructuras > modulos > ✎ GestorUsuarios.pm > {} estructuras::modulos::GestorUsuarios
1  package estructuras::modulos::GestorUsuarios;

94
95 sub existeUsuario {
96     my ($self, $numero_colegio) = @_;
97     return defined $self->buscarUsuario($numero_colegio);
98 }
99
100 # =====
101 # Eliminación
102 # =====
103 sub eliminarUsuario {
104     my ($self, $numero_colegio) = @_;
105
106     if (!defined $numero_colegio || $numero_colegio eq '') {
107         return (0, 'Debe indicar el numero de colegio');
108     }
109
110     return $self->{arbol}->eliminar($numero_colegio);
111 }
112
113 # =====
114 # Login / autenticación
115 # =====
116 sub autenticarUsuario {
117     my ($self, $numero_colegio, $contrasena) = @_;
118
119     if (!defined $numero_colegio || $numero_colegio eq '') {
120         return (0, 'Debe indicar el numero de colegio', undef);
121     }
122
123     if (!defined $contrasena) {
124         return (0, 'Debe indicar la contrasena', undef);
125     }
126
127     my $usuario = $self->buscarUsuario($numero_colegio);
128
129     if (!$usuario) {
130         return (0, 'Usuario no encontrado', undef);
131     }
132
133     if (!$usuario->can('verificarContrasena')) {
134         return (0, 'El modelo de usuario no permite verificar contrasena', undef);
135     }
136
137     if (!$usuario->verificarContrasena($contrasena)) {
138         return (0, 'Contrasena incorrecta', undef);
139     }

```

8.3 GestorPermisos.pm

Define qué puede consultar o solicitar cada usuario según:

- tipo de usuario,
- departamento.

```

estructuras > modulos > ✎ GestorPermisos.pm > {} estructuras::modulos::GestorPermisos
1  package estructuras::modulos::GestorPermisos;
2
3  use strict;
4  use warnings;
5
6  sub new {
7      my ($class, @args) = @_;
8
9      my %args;
10     if (@args == 1 && ref($args[0]) eq 'HASH') {
11         %args = %{ $args[0] };
12     } else {
13         %args = @args;
14     }
15
16     my $self = {
17         tipos_inventario => $args{tipos_inventario} || [qw(MEDICAMENTO EQUIPO SUMINISTRO)],
18         reglas_tipo      => $args{reglas_tipo}      || _reglas_tipo_por_defecto(),
19         reglas_departamento => $args{reglas_departamento} || _reglas_departamento_por_defecto(),
20     };
21
22     bless $self, $class;
23     return $self;
24 }
25
26 # =====
27 # Reglas por defecto
28 # =====
29 sub _reglas_tipo_por_defecto {
30     return {
31         'TIPO-01' => {
32             consulta => [qw(MEDICAMENTO EQUIPO SUMINISTRO)],
33             solicitud => [qw(MEDICAMENTO EQUIPO SUMINISTRO)],
34         },
35         'TIPO-02' => {
36             consulta => [qw(MEDICAMENTO EQUIPO SUMINISTRO)],
37             solicitud => [qw(MEDICAMENTO EQUIPO SUMINISTRO)],
38         },
39         'TIPO-03' => {
40             consulta => [qw(MEDICAMENTO EQUIPO SUMINISTRO)],
41             solicitud => [qw(SUMINISTRO)],
42         },
43         'TIPO-04' => {
44             consulta => [qw(MEDICAMENTO SUMINISTRO)],
45             solicitud => [qw(MEDICAMENTO SUMINISTRO)],
46         },
47         'TIPO-05' => {

```

estructuras > modulos > ✎ GestorPermisos.pm > {} estructuras::modulos::GestorPermisos

```
1  package estructuras::modulos::GestorPermisos;
29 sub _reglas_tipo_por_defecto {
49     solicitud => [],
50 },
51 };
52 }
53
54 sub _reglas_departamento_por_defecto {
55     return {
56         'DEP-MED' => {
57             consulta => [qw(MEDICAMENTO EQUIPO SUMINISTRO)],
58             solicitud => [qw(MEDICAMENTO EQUIPO SUMINISTRO)],
59         },
60         'DEP-CIR' => {
61             consulta => [qw(MEDICAMENTO EQUIPO SUMINISTRO)],
62             solicitud => [qw(MEDICAMENTO EQUIPO SUMINISTRO)],
63         },
64         'DEP-FAR' => {
65             consulta => [qw(MEDICAMENTO SUMINISTRO)],
66             solicitud => [qw(MEDICAMENTO SUMINISTRO)],
67         },
68         'DEP-LAB' => {
69             consulta => [qw(EQUIPO SUMINISTRO)],
70             solicitud => [qw(SUMINISTRO)],
71         },
72         'DEP-ADM' => {
73             consulta => [qw(MEDICAMENTO EQUIPO SUMINISTRO)],
74             solicitud => [],
75         },
76     };
77 }
78
79 # =====
80 # Getters básicos
81 # =====
82 sub getTiposInventario {
83     my ($self) = @_;
84     return $self->{tipos_inventario};
85 }
86
87 sub getReglasTipo {
88     my ($self) = @_;
89     return $self->{reglas_tipo};
90 }
91
92 sub getReglasDepartamento {
93     my ($self) = @_;
```

```

estructuras > modulos > ✎ GestorPermisos.pm > {} estructuras::modulos::GestorPermisos
1  package estructuras::modulos::GestorPermisos;
2  # =====
98 # Obtención de permisos efectivos
99 # =====
100 sub obtenerPermisosUsuario {
101     my ($self, $usuario) = @_;
102
103     my $ctx = $self->_resolver_contexto_usuario($usuario);
104
105     my $rol      = $ctx->{rol};
106     my $tipo     = $ctx->{tipo_usuario};
107     my $departamento = $ctx->{departamento};
108
109     # Admin del sistema: acceso completo
110     if (defined $rol && uc($rol) eq 'ADMIN') {
111         my @todos = @{$self->{tipos_inventario}};
112
113         return {
114             rol      => 'ADMIN',
115             tipo_usuario => $tipo,
116             departamento => $departamento,
117             consulta   => \@todos,
118             solicitud  => \@todos,
119             es_admin   => 1,
120         };
121     }
122
123     my $regla_tipo = $self->{reglas_tipo}{$tipo} || {
124         consulta => [],
125         solicitud => [],
126     };
127
128     my $regla_depto = $self->{reglas_departamento}{$departamento} || {
129         consulta => [],
130         solicitud => [],
131     };
132
133     my @consulta_efectiva = $self->_interseccion($regla_tipo->{consulta}, $regla_depto->{consulta});
134     my @solicitud_efectiva = $self->_interseccion($regla_tipo->{solicitud}, $regla_depto->{solicitud});
135
136     return {
137         rol      => $rol,
138         tipo_usuario => $tipo,
139         departamento => $departamento,
140         consulta   => \@consulta_efectiva,
141         solicitud  => \@solicitud_efectiva,
142         es_admin   => 0,
143     };

```

8.4 Cargador JSON.pm

Se encarga de:

- leer archivos JSON,
- validar estructuras,
- construir objetos,
- insertarlos en sus estructuras correspondientes.

estructuras > modulos > CargadorJSON.pm > {} estructuras::modulos::CargadorJSON

```
1  package estructuras::modulos::CargadorJSON;
2
3  use strict;
4  use warnings;
5
6  use JSON::PP qw(decode_json);
7
8  use modelos::Proveedor;
9  use modelos::Medicamento;
10 use modelos::Equipo;
11 use modelos::Suministro;
12 use modelos::PersonalMedico;
13
14 sub new {
15     my ($class, @args) = @_;
16
17     my %args;
18
19     # Compatibilidad:
20     # 1) new({ ... })
21     # 2) new(clave => valor, ...)
22     if (@args == 1 && ref($args[0]) eq 'HASH') {
23         %args = %{ $args[0] };
24     } else {
25         %args = @args;
26     }
27
28     my $self = {
29         inventario      => $args{inventario},
30         gestor_usuarios => $args{gestor_usuarios},
31     };
32
33     bless $self, $class;
34     return $self;
35 }
36
37 # =====
38 # Getters
39 # =====
40 sub getInventario {
41     my ($self) = @_;
42     return $self->{inventario};
43 }
44
45 sub getGestorUsuarios {
46     my ($self) = @_;
47     return $self->{gestor_usuarios};
```

estructuras > modulos > CargadorJSON.pm > {} estructuras::modulos::CargadorJSON

```
1  package estructuras::modulos::CargadorJSON;
48 }
49
50 # =====
51 # Carga de inventario
52 # =====
53 sub cargarInventarioDesdeArchivo {
54     my ($self, $ruta) = @_;
55
56     if (!defined $self->{inventario}) {
57         return {
58             ok => 0,
59             mensaje => 'No se ha configurado el inventario en el cargador',
60             errores => ['Falta la instancia de Inventario.pm'],
61         };
62     }
63
64     my $data = eval { $self->_leer_json($ruta) };
65     if ($@) {
66         return {
67             ok => 0,
68             mensaje => "No se pudo leer el archivo de inventario: $@",
69             errores => ["Error de lectura/parsing en $ruta"],
70         };
71     }
72
73     if (ref($data) ne 'HASH' || ref($data->{proveedor}) ne 'ARRAY') {
74         return {
75             ok => 0,
76             mensaje => 'El JSON de inventario no tiene la estructura esperada',
77             errores => ['Se esperaba una clave "proveedor" con un arreglo'],
78         };
79     }
80
81     my $resumen = {
82         ok => 1,
83         mensaje => 'Carga de inventario completada',
84         archivo => $ruta,
85         proveedores_leidos => 0,
86         proveedores_registrados => 0,
87         items_leidos => 0,
88         medicamentos_ok => 0,
89         equipos_ok => 0,
90         suministros_ok => 0,
91         errores => [],
92     };
93 }
```

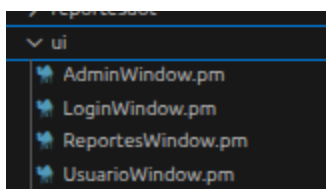
```

estructuras > modulos > CargadorJSON.pm > {} estructuras::modulos::CargadorJSON
1  package estructuras::modulos::CargadorJSON;
53 sub cargarInventarioDesdeArchivo {
95     $resumen->{proveedores_leidos}++;
96
97     if (ref($registro_proveedor) ne 'HASH') {
98         push @{$resumen->{errores}}, "Registro de proveedor invalido en posicion $resumen->{proveedores_leido
99         next;
100    }
101
102    my $nit = $self->_normalizar($registro_proveedor->{nit});
103    my $nombre = $self->_normalizar($registro_proveedor->{nombre});
104    my $telefono = $self->_normalizar($registro_proveedor->{telefono});
105    my $direccion = $self->_normalizar($registro_proveedor->{direccion});
106
107    my $proveedor = modelos::Proveedor->new(
108        nit => $nit,
109        nombre_empresa => $nombre,
110        contacto => 'Carga JSON',
111        telefono => $telefono,
112        direccion => $direccion,
113    );
114
115    my ($ok_prov, $msg_prov) = $self->{inventario}->registrarProveedor($proveedor);
116
117    if ($ok_prov) {
118        $resumen->{proveedores_registrados}++;
119    } else {
120        # si ya existía, seguimos cargando sus items de todos modos
121        push @{$resumen->{errores}}, "Proveedor $nit: $msg_prov";
122    }
123
124    my $entregas = $registro_proveedor->{entrega};
125
126    if (ref($entregas) ne 'ARRAY') {
127        push @{$resumen->{errores}}, "Proveedor $nit no tiene un arreglo valido en 'entrega'";
128        next;
129    }
130
131    foreach my $item (@$entregas) {
132        $resumen->{items_leidos}++;
133
134        if (ref($item) ne 'HASH') {
135            push @{$resumen->{errores}}, "Item invalido en proveedor $nit";
136            next;
137        }
138
139        my $tipo = uc($self->_normalizar($item->{tipo}));

```

9. Interfaz gráfica

La interfaz se implementó con GTK3 y se divide en:



9.1 LoginWindow.pm

Maneja:

- autenticación de administrador,
- autenticación de usuarios,
- navegación hacia ventanas principales,
- pestaña de datos del desarrollador.

```
ui > LoginWindow.pm > {} ui::LoginWindow > _crear_pestana_datos
1  package ui::LoginWindow;
2
3  use strict;
4  use warnings;
5  use utf8;
6
7  use Gtk3;
8  use Glib qw/TRUE FALSE/;
9
10 use ui::AdminWindow;
11 use ui::UsuarioWindow;
12
13 sub new {
14     my ($class, %args) = @_;
15
16     my $self = {
17         gestor_usuarios => $args{gestor_usuarios},
18         gestor_permisos => $args{gestor_permisos},
19         inventario      => $args{inventario},
20         cargador_json   => $args{cargador_json},
21
22         admin_user      => defined $args{admin_user} ? $args{admin_user} : 'admin',
23         admin_password  => defined $args{admin_password} ? $args{admin_password} : 'admin123',
24
25         on_exit         => $args{on_exit},
26
27         # Datos personales / académicos
28         datos_personales => $args{datos_personales} || {
29             nombre      => 'Tu nombre aquí',
30             carnet      => 'Tu carnet aquí',
31             curso       => 'Estructuras de Datos',
32             seccion     => 'Sección X',
33             proyecto    => 'EDD MedTrack',
34             fase        => 'Fase 2',
35             semestre    => '2026',
36         },
37
38         window          => undef,
39         entry_usuario   => undef,
40         entry_password  => undef,
41         lbl_estado      => undef,
42
43         admin_window    => undef,
44         usuario_window  => undef,
45     };
46
47     bless $self, $class;
```

```

ui > LoginWindow.pm > {} ui::LoginWindow > _crear_pestana_datos
1  package ui::LoginWindow;
13 sub new {
49
50     return $self;
51 }
52
53 # =====
54 # Construcción de interfaz
55 # =====
56 sub _build_ui {
57     my ($self) = @_;
58
59     my $window = Gtk3::Window->new('toplevel');
60     $window->set_title('EDD MedTrack - Login');
61     $window->set_default_size(520, 360);
62     $window->set_border_width(12);
63     $window->set_resizable(FALSE);
64
65     $window->signal_connect(
66         destroy => sub {
67             if (defined $self->{on_exit} && ref($self->{on_exit}) eq 'CODE') {
68                 $self->{on_exit}->();
69             } else {
70                 Gtk3::main_quit();
71             }
72         }
73     );
74
75     my $main_vbox = Gtk3::Box->new('vertical', 10);
76     $window->add($main_vbox);
77
78     my $lbl_app = Gtk3::Label->new();
79     $lbl_app->set_markup('<span size="x-large" weight="bold">EDD MedTrack</span>');
80     $lbl_app->set_xalign(0);
81
82     my $lbl_desc = Gtk3::Label->new(
83         'Sistema hospitalario con estructuras no lineales, carga JSON y visualización de reportes.'
84     );
85     $lbl_desc->set_xalign(0);
86     $lbl_desc->set_line_wrap(TRUE);
87
88     $main_vbox->pack_start($lbl_app, FALSE, FALSE, 0);
89     $main_vbox->pack_start($lbl_desc, FALSE, FALSE, 0);
90
91     my $notebook = Gtk3::Notebook->new();
92     $main_vbox->pack_start($notebook, TRUE, TRUE, 0);
93

```

```

ui > LoginWindow.pm > {} ui::LoginWindow > _crear_pestana_datos
1  package ui::LoginWindow;
56 sub _build_ui {
95     # Pestaña 1: Login
96     # =====
97     my $pagina_login = Gtk3::Box->new('vertical', 12);
98     $pagina_login->set_border_width(12);
99
100     my $lbl_titulo = Gtk3::Label->new();
101     $lbl_titulo->set_markup('<span size="large" weight="bold">Inicio de sesión</span>');
102     $lbl_titulo->set_xalign(0);
103
104     my $lbl_subtitulo = Gtk3::Label->new(
105         'Admin: usa credenciales del sistema. Usuario: usa número de colegio y contraseña.'
106     );
107     $lbl_subtitulo->set_xalign(0);
108     $lbl_subtitulo->set_line_wrap(TRUE);
109
110     $pagina_login->pack_start($lbl_titulo, FALSE, FALSE, 0);
111     $pagina_login->pack_start($lbl_subtitulo, FALSE, FALSE, 0);
112
113     my $grid = Gtk3::Grid->new();
114     $grid->set_row_spacing(10);
115     $grid->set_column_spacing(10);
116
117     my $lbl_usuario = Gtk3::Label->new('Usuario / No. Colegio:');
118     $lbl_usuario->set_xalign(0);
119
120     my $entry_usuario = Gtk3::Entry->new();
121     $entry_usuario->set_hexpand(TRUE);
122     $entry_usuario->set_placeholder_text('Ej. admin o COL-10001');
123
124     my $lbl_password = Gtk3::Label->new('Contraseña:');
125     $lbl_password->set_xalign(0);
126
127     my $entry_password = Gtk3::Entry->new();
128     $entry_password->set_visibility(FALSE);
129     $entry_password->set_invisible_char('*');
130     $entry_password->set_hexpand(TRUE);
131     $entry_password->set_placeholder_text('Ingrese su contraseña');
132
133     $grid->attach($lbl_usuario, 0, 0, 1, 1);
134     $grid->attach($entry_usuario, 1, 0, 1, 1);
135     $grid->attach($lbl_password, 0, 1, 1, 1);
136     $grid->attach($entry_password, 1, 1, 1, 1);
137
138     $pagina_login->pack_start($grid, FALSE, FALSE, 0);
139

```

9.2 AdminWindow.pm

Permite al administrador:

- cargar JSON,
- gestionar equipos,
- gestionar suministros,
- gestionar usuarios,
- consultar proveedor/fabricante,

- visualizar reportes.

```

ui > AdminWindow.pm > {} ui::AdminWindow
1  package ui::AdminWindow;
2
3  use strict;
4  use warnings;
5  use utf8;
6
7  use Gtk3;
8  use Glib qw/TRUE FALSE/;
9
10 use modelos::Equipo;
11 use modelos::Suministro;
12 use modelos::PersonalMedico;
13
14 sub new {
15     my ($class, %args) = @_;
16
17     my $self = {
18         inventario      => $args{inventario},
19         gestor_usuarios => $args{gestor_usuarios},
20         cargador_json   => $args{cargador_json},
21
22         on_logout       => $args{on_logout},
23
24         path_reportes   => defined $args{path_reportes} ? $args{path_reportes} : 'reportesdot',
25
26         window          => undef,
27         textview_log     => undef,
28         textbuffer_log   => undef,
29         lbl_bienvenida   => undef,
30
31         image_reporte    => undef,
32         lbl_reporte      => undef,
33     };
34
35     bless $self, $class;
36     $self->_build_ui();
37
38     return $self;
39 }
40
41 # =====
42 # Construcción de interfaz
43 # =====
44 sub _build_ui {
45     my ($self) = @_;
46
47     my $window = Gtk3::Window->new('toplevel');

```

```

ui > AdminWindow.pm > {} ui::AdminWindow
1  package ui::AdminWindow;
44 sub _build_ui {
49     $window->set_default_size(1380, 860);
50     $window->set_border_width(12);
51
52     $window->signal_connect(
53         destroy => sub {
54             Gtk3::main_quit();
55         }
56     );
57
58     my $main_hbox = Gtk3::Box->new('horizontal', 12);
59     $window->add($main_hbox);
60
61     my $left_vbox = Gtk3::Box->new('vertical', 10);
62     $main_hbox->pack_start($left_vbox, TRUE, TRUE, 0);
63
64     my $lbl_titulo = Gtk3::Label->new();
65     $lbl_titulo->set_markup('<span size="x-large" weight="bold">Panel de Administración</span>');
66     $lbl_titulo->set_xalign(0);
67
68     my $lbl_bienvenida = Gtk3::Label->new('Bienvenido, administrador del sistema.');
```

```

69     $lbl_bienvenida->set_xalign(0);
70
71     $left_vbox->pack_start($lbl_titulo, FALSE, FALSE, 0);
72     $left_vbox->pack_start($lbl_bienvenida, FALSE, FALSE, 0);
73
74     my $frame_acciones = Gtk3::Frame->new('Acciones disponibles');
```

```

75     $left_vbox->pack_start($frame_acciones, FALSE, FALSE, 0);
76
77     my $grid = Gtk3::Grid->new();
78     $grid->set_row_spacing(8);
79     $grid->set_column_spacing(8);
80     $grid->set_border_width(8);
81     $frame_acciones->add($grid);
82
83     my $btn_cargar_inventario = Gtk3::Button->new('Cargar inventario JSON');
```

```

84     my $btn_cargar_usuarios = Gtk3::Button->new('Cargar usuarios JSON');
```

```

85     my $btn_cargar_todo = Gtk3::Button->new('Cargar ambos JSON');
```

```

86     my $btn_generar_reportes = Gtk3::Button->new('Generar reportes');
```

```

87     my $btn_resumen = Gtk3::Button->new('Ver resumen del sistema');
```

```

88     my $btn_logout = Gtk3::Button->new('Cerrar sesión');
```

```

89
90     my $btn_registrar_equipo = Gtk3::Button->new('Registrar equipo');
```

```

91     my $btn_buscar_equipo = Gtk3::Button->new('Buscar equipo');
```

```

92     my $btn_editar_equipo = Gtk3::Button->new('Editar equipo');
```

```

93     my $btn_eliminar_equipo = Gtk3::Button->new('Eliminar equipo');
```

```

ui > AdminWindow.pm > {} ui::AdminWindow
1  package ui::AdminWindow;
44  sub build_ui {
98      my $btn_editar_suministro = Gtk3::Button->new('Editar suministro');
99      my $btn_eliminar_suministro = Gtk3::Button->new('Eliminar suministro');
100     my $btn_recorrido_suministro = Gtk3::Button->new('Ver suministros por recorrido');
101
102     my $btn_registrar_usuario = Gtk3::Button->new('Registrar usuario');
103     my $btn_buscar_usuario = Gtk3::Button->new('Buscar usuario');
104     my $btn_eliminar_usuario = Gtk3::Button->new('Eliminar usuario');
105     my $btn_recorrido_usuario = Gtk3::Button->new('Ver usuarios por recorrido');
106     my $btn_tabla_usuarios = Gtk3::Button->new('Tabla personal médico');
107
108     my $btn_consultar_pf = Gtk3::Button->new('Consultar Prov/Fab');
109     my $btn_comparar_pf = Gtk3::Button->new('Comparar Prov/Fab');
110
111     my $btn_ver_bst = Gtk3::Button->new('Ver reporte BST');
112     my $btn_ver_avl = Gtk3::Button->new('Ver reporte AVL');
113     my $btn_ver_b = Gtk3::Button->new('Ver reporte Árbol B');
114     my $btn_ver_matriz = Gtk3::Button->new('Ver reporte Matriz');
115
116     $grid->attach($btn_cargar_inventario, 0, 0, 1, 1);
117     $grid->attach($btn_cargar_usuarios, 1, 0, 1, 1);
118     $grid->attach($btn_cargar_todo, 2, 0, 1, 1);
119
120     $grid->attach($btn_generar_reportes, 0, 1, 1, 1);
121     $grid->attach($btn_resumen, 1, 1, 1, 1);
122     $grid->attach($btn_logout, 2, 1, 1, 1);
123
124     $grid->attach($btn_registrar_equipo, 0, 2, 1, 1);
125     $grid->attach($btn_buscar_equipo, 1, 2, 1, 1);
126     $grid->attach($btn_editar_equipo, 2, 2, 1, 1);
127
128     $grid->attach($btn_eliminar_equipo, 0, 3, 1, 1);
129     $grid->attach($btn_recorrido_equipo, 1, 3, 2, 1);
130
131     $grid->attach($btn_registrar_suministro, 0, 4, 1, 1);
132     $grid->attach($btn_buscar_suministro, 1, 4, 1, 1);
133     $grid->attach($btn_editar_suministro, 2, 4, 1, 1);
134
135     $grid->attach($btn_eliminar_suministro, 0, 5, 1, 1);
136     $grid->attach($btn_recorrido_suministro, 1, 5, 2, 1);
137
138     $grid->attach($btn_registrar_usuario, 0, 6, 1, 1);
139     $grid->attach($btn_buscar_usuario, 1, 6, 1, 1);
140     $grid->attach($btn_eliminar_usuario, 2, 6, 1, 1);
141
142     $grid->attach($btn_recorrido_usuario, 0, 7, 2, 1);
143

```

9.3 UsuarioWindow.pm

Permite al personal médico:

- ver inventario según permisos,
- buscar medicamentos,
- revisar alertas,
- consultar disponibilidad.

```

ui >  UsuarioWindow.pm > {} ui::UsuarioWindow >  get_window
1  package ui::UsuarioWindow;
2
3  use strict;
4  use warnings;
5  use utf8;
6
7  use Gtk3;
8  use Glib qw/TRUE FALSE/;
9  use Time::Piece;
10 use Time::Seconds;
11
12 sub new {
13     my ($class, %args) = @_;
14
15     my $self = {
16         inventario    => $args{inventario},
17         gestor_permisos => $args{gestor_permisos},
18         usuario       => $args{usuario},
19         on_logout     => $args{on_logout},
20
21         window        => undef,
22
23         lbl_bienvenida => undef,
24         lbl_permisos  => undef,
25
26         lbl_med_count => undef,
27         lbl_equ_count => undef,
28         lbl_sum_count => undef,
29         lbl_alert_stock => undef,
30         lbl_alert_venc => undef,
31
32         store_general => undef,
33         store_busqueda => undef,
34         store_alertas => undef,
35
36         entry_busqueda => undef,
37         combo_campo    => undef,
38         check_critico  => undef,
39         check_vencen   => undef,
40     };
41
42     bless $self, $class;
43     $self->_build_ui();
44     $self->_refrescar_todo();
45
46     return $self;
47 }

```

```

ui > UsuarioWindow.pm > {} ui::UsuarioWindow > get_window
1  package ui::UsuarioWindow;
48
49  # =====
50  # Construcción de interfaz
51  # =====
52  sub _build_ui {
53      my ($self) = @_;
54
55      my $window = Gtk3::Window->new('toplevel');
56      $window->set_title('EDD MedTrack - Panel de Usuario');
57      $window->set_default_size(1280, 820);
58      $window->set_border_width(12);
59
60      $window->signal_connect(
61          destroy => sub {
62              if (defined $self->{on_logout} && ref($self->{on_logout}) eq 'CODE') {
63                  $self->{on_logout}->();
64              } else {
65                  Gtk3::main_quit();
66              }
67          }
68      );
69
70      my $main_vbox = Gtk3::Box->new('vertical', 10);
71      $window->add($main_vbox);
72
73      # =====
74      # Encabezado
75      # =====
76      my $lbl_titulo = Gtk3::Label->new();
77      $lbl_titulo->set_markup('<span size="x-large" weight="bold">Panel del Personal Médico</span>');
78      $lbl_titulo->set_xalign(0);
79
80      my $lbl_bienvenida = Gtk3::Label->new('Bienvenido. ');
81      $lbl_bienvenida->set_xalign(0);
82
83      my $lbl_permisos = Gtk3::Label->new('Permisos: ');
84      $lbl_permisos->set_xalign(0);
85      $lbl_permisos->set_line_wrap(TRUE);
86
87      $main_vbox->pack_start($lbl_titulo, FALSE, FALSE, 0);
88      $main_vbox->pack_start($lbl_bienvenida, FALSE, FALSE, 0);
89      $main_vbox->pack_start($lbl_permisos, FALSE, FALSE, 0);
90
91      # =====
92      # Resumen rápido
93      # =====

```



```

ui > UsuarioWindow.pm > {} ui::UsuarioWindow > get_window
1  package ui::UsuarioWindow;
52  sub build_ui {
91  # =====
92  # Resumen rápido
93  # =====
94  my $frame_resumen = Gtk3::Frame->new('Vista general del inventario');
95  $main_vbox->pack_start($frame_resumen, FALSE, FALSE, 0);
96
97  my $grid_resumen = Gtk3::Grid->new();
98  $grid_resumen->set_row_spacing(8);
99  $grid_resumen->set_column_spacing(12);
100 $grid_resumen->set_border_width(8);
101 $frame_resumen->add($grid_resumen);
102
103 my $lbl_med_count = Gtk3::Label->new('Medicamentos visibles: 0');
104 my $lbl_equ_count = Gtk3::Label->new('Equipos visibles: 0');
105 my $lbl_sum_count = Gtk3::Label->new('Suministros visibles: 0');
106 my $lbl_alert_stock = Gtk3::Label->new('Stock crítico: 0');
107 my $lbl_alert_venc = Gtk3::Label->new('Próximos a vencer: 0');
108
109 $lbl_med_count->set_xalign(0);
110 $lbl_equ_count->set_xalign(0);
111 $lbl_sum_count->set_xalign(0);
112 $lbl_alert_stock->set_xalign(0);
113 $lbl_alert_venc->set_xalign(0);
114
115 $grid_resumen->attach($lbl_med_count, 0, 0, 1, 1);
116 $grid_resumen->attach($lbl_equ_count, 1, 0, 1, 1);
117 $grid_resumen->attach($lbl_sum_count, 2, 0, 1, 1);
118 $grid_resumen->attach($lbl_alert_stock, 0, 1, 1, 1);
119 $grid_resumen->attach($lbl_alert_venc, 1, 1, 1, 1);
120
121 # =====
122 # Barra de acciones
123 # =====
124 my $acciones_box = Gtk3::Box->new('horizontal', 10);
125 $main_vbox->pack_start($acciones_box, FALSE, FALSE, 0);
126
127 my $btn_actualizar = Gtk3::Button->new('Actualizar vista');
128 my $btn_logout = Gtk3::Button->new('Cerrar sesión');
129
130 $acciones_box->pack_start($btn_actualizar, FALSE, FALSE, 0);
131 $acciones_box->pack_start($btn_logout, FALSE, FALSE, 0);
132
133 # =====
134 # Notebook principal
135 # =====

```

10. Flujo del sistema

1. el sistema inicia desde `main_fase2.pl`
2. se instancian módulos principales
3. se cargan archivos JSON si existen
4. se abre `LoginWindow`
5. según el tipo de login:
 - admin abre `AdminWindow`
 - usuario abre `UsuarioWindow`

6. al cerrar sesión se retorna al login

11. Carga masiva de datos

El sistema soporta dos archivos JSON:

- inventario masivo
- usuarios departamentales

El cargador:

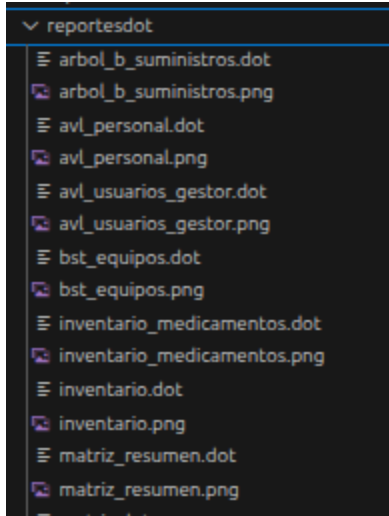
- valida campos requeridos,
- reporta errores,
- inserta registros válidos en sus estructuras.

12. Reportes Graphviz

Los reportes se generan a partir de archivos `.dot` y luego se convierten a `.png`.

Ubicación habitual:

- `reportesdot/bst_equipos.dot`
- `reportesdot/bst_equipos.png`
- `reportesdot/avl_personal.dot`
- `reportesdot/avl_personal.png`
- `reportesdot/arbol_b_suministros.dot`
- `reportesdot/arbol_b_suministros.png`
- `reportesdot/matriz_resumen.dot`
- `reportesdot/matriz_resumen.png`



13. Manejo de errores

El sistema contempla validaciones para:

- campos vacíos,
- códigos duplicados,
- usuarios inválidos,
- errores de lectura JSON,
- estructuras con datos inconsistentes,
- reportes no encontrados,
- credenciales incorrectas.

Esto es importante porque la hoja también penaliza no implementar manejo adecuado de errores.

14. Requisitos de ejecución

Para ejecutar correctamente el sistema:

1. instalar Perl
2. instalar GTK3 para Perl
3. instalar Graphviz
4. ubicar los JSON en la carpeta `cargas`
5. ejecutar `main_fase2.pl`

15. Mantenimiento y extensibilidad

El sistema puede ampliarse agregando:

- edición de perfil de usuario,
- solicitudes formales de inventario,
- persistencia en base de datos,
- nuevos reportes,
- más reglas de permisos.

16. Conclusión

EDD MedTrack integra estructuras de datos no lineales con una aplicación funcional en Perl y GTK3. El sistema cumple con la administración de inventario hospitalario, control de usuarios, consultas diferenciadas y reportes gráficos.